

Matteo Lo Giudice

Professor Jean- Yves Hervé

CSC 406 – Computer Graphics

6 November 2023

Programming Assignment 3, Fall 2023 – Report

ASTEROIDS: A PIONEER IN COMPUTER GRAPHICS AND GAMING

"Asteroids" is a classic arcade game that was developed and published by Atari, Inc. in 1979. It is one of the earliest and most iconic video games, known for its vector graphics and engaging gameplay. The game was designed by Lyle Rains and programmed by Ed Logg, who sought to create a space-themed game that was both challenging and entertaining.

The game is set in the vastness of outer space, where the player controls a small triangular spaceship navigating through a field of drifting asteroids. The objective is simple yet captivating: avoid colliding with asteroids while shooting them to break them into smaller pieces. The player's spaceship is equipped with a laser cannon to shoot the asteroids, and the ability to thrust forward and rotate to navigate the chaotic asteroid field.

"Asteroids" employs vector graphics, which were a novelty at the time. Unlike raster graphics, which use pixels to create images, vector graphics use lines drawn between points. This allowed for smoother animations and a distinctive visual style that set "Asteroids" apart from other games of its era. The game's minimalist black and white graphics, complemented by the eerie silence of space, created an immersive experience for players.

The gameplay mechanics of "Asteroids" are straightforward. The player uses a joystick to rotate the spaceship and buttons to fire the laser cannon, thrust forward, and engage hyperspace for quick escapes. The asteroids, when shot, break into smaller pieces, increasing the difficulty as the game progresses. Additionally, the player must also contend with occasional flying saucers that shoot back.

"Asteroids" was a commercial success and became immensely popular in arcades worldwide. Its innovative use of vector graphics and compelling gameplay contributed to the golden age of arcade video games. The game has since been ported to various platforms and has inspired countless other games and media.

DESIGN CHOICES AND FUNCTIONALITY OF MY ASTEROIDS GAME

In crafting my rendition of the classic "Asteroids" game, I introduced subtle yet impactful design modifications to enhance user engagement and gameplay dynamics. Firstly, I reimagined the spaceship's movement controls. To propel the spaceship forward, players simply press the '2' key. For rotational movement, allowing the spaceship to pivot 360 degrees around its center, I implemented an interactive and dynamic control: the spaceship's direction is determined by the position of the mouse cursor on the screen, ensuring that the ship's tip always points towards the cursor.

This design choice was motivated by a desire to create a more immersive and responsive gaming experience. The direct correlation between the mouse cursor's position and the spaceship's orientation offers players intuitive and real-time control over their in-game avatar.

In addition to the classic asteroid obstacles, I introduced an alien spaceship that fires laser balls towards the player's ship. This added layer of complexity not only increases the game's difficulty but also injects an element of excitement. Players must now strategically navigate through space, avoiding both the asteroids and the alien's laser attacks.

To make the game more player friendly, I incorporated on-screen indicators displaying the player's remaining lives and current score, which increases by eliminating asteroids and alien spaceships. The asteroids in my game are continually generated by the system. Upon destroying an asteroid, there is a probability-based calculation to determine whether a new asteroid will spawn. Additionally, when an asteroid is hit, it fragments into multiple smaller asteroids, adding to the challenge.

A significant design choice was the use of .tga images to represent all game objects and entities. Opting for pre-designed images, as opposed to manually creating each element, allowed for a visually appealing and captivating game environment. This choice aligns with the principle of prioritizing texture over complex shapes, ensuring that the game is not only aesthetically pleasing but also efficiently rendered.

ANALYSIS OF COLLISION DETECTION FUNCTIONS IN THE GAME

In the context of the "Asteroids" game, collision detection is a pivotal aspect that dictates the interaction among various entities within the game. Each entity, such as the alien gun, bullets, asteroids, and the player's spaceship, implements a “*CheckCollision*” function to check if it has collided with another object.

- **AlienGun:**

The *CheckCollision* function in the AlienGun class iterates through all the bullets fired by the alien weapon. For each bullet, it checks if there has been a collision with an object at coordinates (x, y) and of size “*size*”. If a collision is detected, the corresponding bullet is removed from the bullets array, and the function returns true, indicating that a collision has occurred.

- **Bullet:**

The *CheckCollision* function in the Bullet class calculates the distance between the bullet and an object at coordinates (x1, y1) and of size “*size*”. If the distance is less than the sum of the bullet's size (*bulletSize*) and the object's size, the function returns true, signaling a collision.

- **Asteroid:**

Similar to the Bullet class, the CheckCollision function in the Asteroid class calculates the distance between the asteroid and an object at coordinates (x1, y1) and of size “size1”. If the distance is less than the sum of the asteroid's size (size) and the object's size, the function returns true.

- **SpaceShip:**

The CheckCollision function in the SpaceShip class checks if the player's spaceship has collided with an object at coordinates (x1, y1) and of size “size”. It calculates the distance between the spaceship and the object, and if it is less than the sum of the spaceship's size (spaceShipSize) and the object's size, it activates the smoke effect, resets the spaceship's position, and returns true. Additionally, the function also checks for a collision with bullets fired from the spaceship's gun.

- **Rocket:**

1. **Asteroid Collision Check:** The Update function checks for collisions between the rocket and asteroids. It calls asteroidManager → CheckCollision(x, y, rocketSize, true) to check if the rocket, positioned at (x, y) with a size of rocketSize, has collided with any asteroids. If a collision is detected, the hits counter is incremented.
2. **SpaceShip Collision Check:** The same Update function checks for collisions between the rocket and a spaceship. It calls ship → CheckCollision(x, y, rocketSize) to check if the rocket has collided with the spaceship. If a collision is detected, the hits counter is again incremented.
3. **Handling Collisions:** After the collision checks, the Update function iterates hits times to reduce the rocket's health by calling ReduceHealth(ship). This function decreases the livesLeft counter. If livesLeft drops below zero, the gameOver flag is set to true,

indicating that the game is over. Otherwise, smoke is activated at the collision location by calling `smoke.Activate(x, y, -1)`.

Analysis:

These collision detection functions are crucial for ensuring a smooth and dynamic gaming experience. By employing the principle of Euclidean distance, the functions determine whether two objects overlap in the game space. Efficient collision handling, such as removing bullets after a collision, ensures that system resources are optimally utilized. Moreover, the implementation of visual effects, like smoke, enriches the player's visual experience.

BOUNDING BOXES AND KEY FUNCTIONALITIES

In the realm of video games, the concept of **Bounding Boxes** and their associated functionalities is paramount for creating an interactive and dynamic environment. A bounding box is a virtual rectangle that encases an object, simplifying the process of collision detection. By checking if these boxes overlap, the game can quickly determine if two objects, such as spaceships, bullets, or asteroids, have collided. Each game entity, regardless of its complexity, utilizes bounding boxes to streamline interactions and enhance performance.

Key Functionalities pertain to the actions and behaviors associated with each game object. These include movement, shooting, updating states, and rendering visuals on the screen. For instance, a spaceship may have functionalities to navigate, shoot, and respond to collisions. Similarly, bullets and asteroids would have mechanisms to move, detect collisions, and possibly cause damage or be destroyed.

Together, bounding boxes and key functionalities form the backbone of game development, ensuring smooth gameplay, accurate interactions, and an engaging player experience, all while optimizing computational resources.

- AlienGun

The AlienGun class is responsible for managing the firing mechanism of the alien weapon in the game. It maintains a list of bullets, each represented by an instance of the Bullet class, and provides functionalities for adding, updating, drawing, and checking collisions of these bullets.

1. **Bounding Box:** Each bullet fired from the alien gun can be considered to have a bounding box, which is a hypothetical rectangle that encloses the bullet's shape. The bounding box is used to simplify the collision detection process. In the context of the AlienGun class, the bounding box of each bullet is determined by its position (x, y) and its size.
2. **Initialization and Reset:** The Init function initialize the bullet image, while the Reset function clears all the bullets from the list, effectively resetting the state of the alien gun.
3. **Bullet Management:** The Add function adds a new bullet to the list of bullets with a specified trajectory determined by the start (x1, y1) and end (x2, y2) points. The Update function iterates through the list of bullets and updates their positions.
4. **Drawing:** The Draw function activates the bullet image and iterates through the list of bullets to draw each one of them on the screen.
5. **Shooting Mode:** The “SetShooting” function allows toggling the shooting mode of the alien gun, determining whether it can fire bullets or not.

In summary, the AlienGun class efficiently manages the bullets by maintaining their bounding boxes and checking for collisions. The bounding boxes simplify the process of detecting overlaps between bullets and other game entities. By encapsulating functionalities like adding, updating, drawing, and collision detection, the AlienGun class plays a crucial role in the game's dynamics and interactions.

- **Bullet**

In the context of game development, the Bullet class is a quintessential example of how Bounding Boxes and Key Functionalities seamlessly work together to create an immersive gaming experience. The Bullet class encapsulates the properties and behaviors of bullets within the game, ensuring that they interact correctly with other entities.

1. **Bounding Boxes:** Each bullet is represented by a bounding box, defined by its size (bulletSize). This invisible rectangle surrounds the bullet, simplifying the process of collision detection and ensuring that interactions with other game objects are computed efficiently.
2. **Key Functionalities:** Beyond collision detection, the Bullet class is endowed with several functionalities:
 - **Initialization:** The constructor initializes the bullet's position (x, y), direction (dx, dy), and sets it as active. The direction is normalized to ensure consistent speed across different trajectories.
 - **Movement:** The Update function updates the bullet's position based on its direction and speed, ensuring that it travels smoothly across the screen. It also deactivates the bullet if it moves outside the game area, optimizing resources.
 - **Rendering:** The Draw function visually renders the bullet on the screen, using its bounding box dimensions to place it accurately within the game space.

Together, these elements ensure that the bullet behaves realistically, contributing to a dynamic and engaging gameplay experience.

- **Rocket**

The Rocket class in the game is a comprehensive representation of the player's spacecraft, encapsulating its behavior, movement, and interactions within the game environment. It is

designed with Bounding Boxes and several Key Functionalities to ensure a dynamic and responsive gameplay experience.

1. **Bounding Boxes:** The Rocket class uses a bounding box defined by `rocketSize` to represent the spacecraft. This bounding box simplifies collision detection and ensures that the rocket's interactions with other game entities, such as asteroids and the spaceship, are accurately computed.
2. **Key Functionalities:** The Rocket class is endowed with a variety of functionalities:
 - **Initialization and Relocation:** The `Init` and `Relocate` functions initialize and reposition the rocket, setting up textures, smoke effects, and missile management.
 - **Movement and Rotation:** The `Update` function updates the rocket's position based on its speed and direction (`theta`). It also checks for collisions and adjusts the rocket's position to ensure it stays within the game area.
 - **Shooting Mechanism:** The `Shoot` function allows the rocket to fire missiles, while the `missileManager` manages these missiles' trajectories and interactions.
 - **Rendering:** The `Draw` function renders the rocket, its missiles, and visual effects like smoke and fire on the screen. The `DrawLives` function displays the remaining lives of the rocket.
 - **Health Management:** The `ReduceHealth` function manages the rocket's health, updating the game state and activating visual effects upon collisions.

In summary, the Rocket class, through its bounding boxes and key functionalities, ensures a seamless and interactive gaming experience, allowing players to navigate, shoot, and respond to in-game events effectively.

- AsteroidManager

The AsteroidManager class plays a pivotal role in the game by managing the lifecycle, behavior, and interactions of asteroids within the game environment. It employs Bounding

Boxes and several Key Functionalities to ensure a dynamic and immersive gameplay experience.

1. **Bounding Boxes:** The AsteroidManager class uses bounding boxes to represent each asteroid in the game. These bounding boxes simplify the process of collision detection and ensure accurate interactions between asteroids and other game entities.
2. **Key Functionalities:** The AsteroidManager class is equipped with several functionalities:
 - **Initialization and Destruction:** The Init function initializes the asteroids, setting up textures and creating a linked list of asteroid objects. The Destroy function deallocates memory and cleans up the list of asteroids.
 - **Asteroid Management:** The Update function iterates through the linked list of asteroids, updating their positions and states. The Draw function renders the asteroids on the screen.
 - **Asteroid Removal and Addition:** The Remove function removes an asteroid from the linked list and optionally adds a new asteroid based on a probability determined by the golden ratio.

In summary, the AsteroidManager class, through its use of bounding boxes and key functionalities, ensures that asteroids are effectively managed, displayed, and interacted with in the game. It contributes to creating a dynamic and challenging environment, enhancing the overall gaming experience.

- Asteroid

The Asteroid class is crucial in simulating the behavior of asteroids within the game, making use of Bounding Boxes and several Key Functionalities to create a dynamic and interactive gameplay experience.

1. **Bounding Boxes:** Each asteroid is represented by a bounding box, simplifying the process of collision detection. These bounding boxes ensure that interactions between asteroids and other game entities are accurately detected and handled.
2. **Key Functionalities:** The Asteroid class encompasses several functionalities:
 - **Initialization and Resetting:** The Asteroid constructor and Reset function initialize or reset the properties of an asteroid, such as its position, direction, size, and color, ensuring variety in the game.
 - **Asteroid Movement:** The Update function updates the position and rotation of the asteroid based on its speed and direction. It also checks if the asteroid has moved out of the game area and resets it accordingly.
 - **Rendering:** The Draw function renders the asteroid on the screen with the appropriate color and size.
 - **Cloning:** The Clone function creates a new asteroid when an existing one is hit, dividing its size by two and changing its direction. This simulates the breaking of asteroids upon collision.

In essence, the Asteroid class, through its use of bounding boxes and key functionalities, ensures that asteroids are effectively simulated, displayed, and interacted with in the game.

- **Missile**

The Missile class is integral to the game's combat mechanics, employing Bounding Boxes and several Key Functionalities to simulate the behavior of missiles fired within the game.

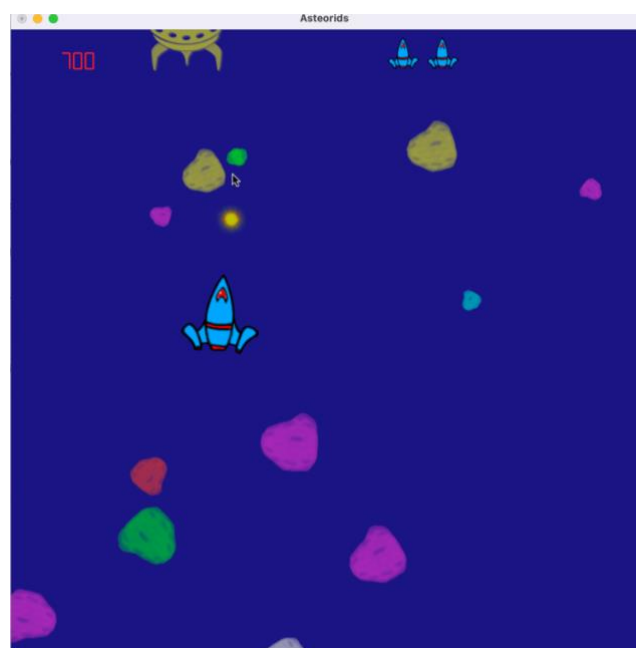
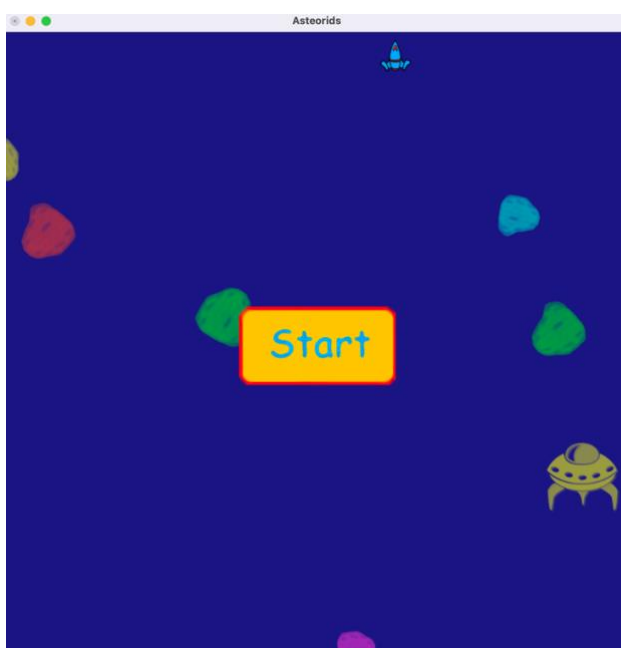
1. **Bounding Boxes:** Each missile is encapsulated within a bounding box, which is pivotal for detecting collisions with other game entities such as asteroids and spaceships. This ensures that the game can accurately determine when and where interactions occur.
2. **Key Functionalities:** The Missile class is equipped with several functionalities:

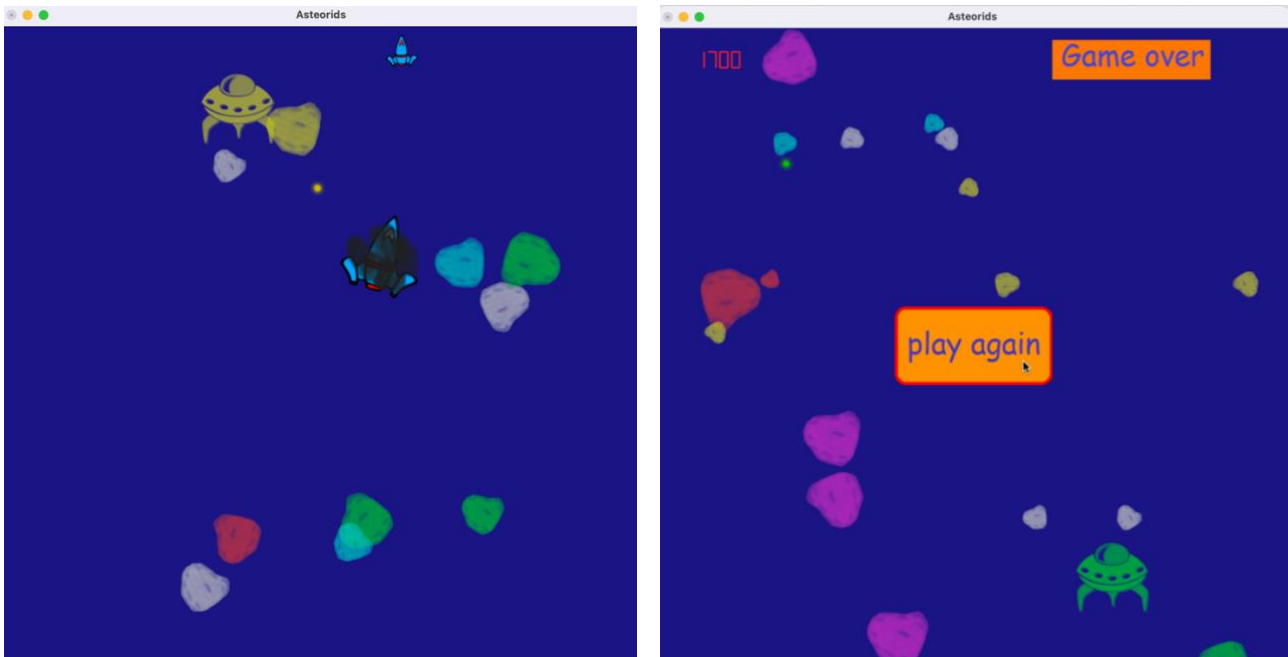
- **Initialization:** The Missile constructor initializes the missile's properties, such as its position, direction, and pointers to the next and previous missiles in a linked list.
- **Missile Movement:** The Update function updates the missile's position based on its speed and direction. It also checks for collisions with asteroids and spaceships, updating the score accordingly, and determines if the missile has moved out of the game area.
- **Rendering:** The Draw function is responsible for rendering the missile on the screen, ensuring that the player can see the missile's trajectory.

In summary, the Missile class, through its use of bounding boxes and key functionalities, ensures that missiles are effectively simulated, displayed, and interacted with in the game.

SCREENSHOTS

I would like to include a few screenshots of the game to give a better visual understanding of the gameplay and graphics. These images will provide a glimpse into the game's environment, characters, and interactions.



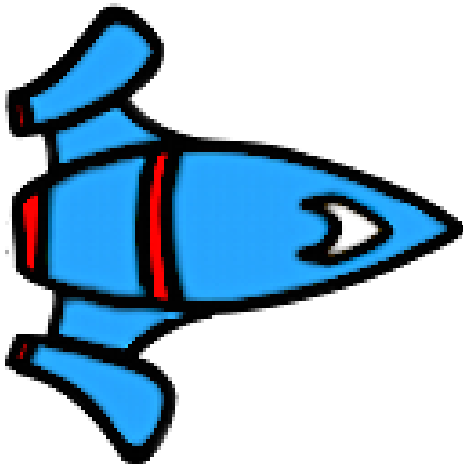


EXTRA CREDITS IMPLEMENTED

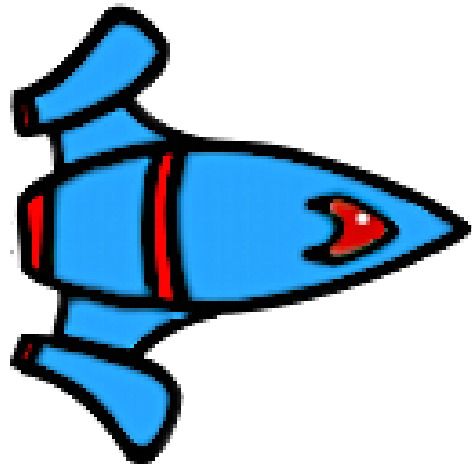
Throughout the development of this project, I have taken special care to incorporate several advanced features that go beyond the basic requirements, aiming to secure extra credits. In the following sections, I will detail how I implemented these standout elements, which not only enhance the gaming experience but also showcase an extra layer of commitment in my work.

- **5.1 Extra credit 1**

I have carefully selected an image for my spaceship that is visually appealing while also aligning well with the bounding boxes, ensuring a seamless integration into the game without the need for complex modeling.



ROCKET



ROCKET FIRING

- **5.2 + 5.3 Extra credit 2 + Extra credit 3**

I have implemented the feature to display the score, number of remaining lives. This enhancement is clearly showcased in the screenshots provided above.

- **5.4 Extra credit 4**

I have also implemented this feature, where exhaust flames are displayed when thrust is applied to the spaceship.

